

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

УДК 519.8

СОГЛАСОВАНО

Генеральный директор
АО "Центр открытых систем
и высоких технологий"

А.М. Данилов

УТВЕРЖДАЮ

Академический руководитель
образовательной программы
«Программная инженерия»
старший преподаватель
департамента программной
инженерии

Н.А. Павлочев

Отчет

по исследовательскому курсовому проекту

на тему **МОДЕЛЬ ДЛЯ ЗАДАЧИ СИТУАЦИОННОЙ ОСВЕДОМЛЕННОСТИ НА
ПРИМЕРЕ МОДИФИЦИРОВАННОЙ ЗАДАЧИ ЭЙНШТЕЙНА**

по направлению подготовки бакалавров 09.03.04 «Программная инженерия»

Выполнил
студент группы БПИ 231
образовательной программы
09.03.04 «Программная инженерия»

К.И. Дик

24.04.2026

Москва 2026

РЕФЕРАТ

Отчет 20 с., 10 источн., 1 прил.

МОДЕЛЬ ДЛЯ ЗАДАЧИ СИТУАЦИОННОЙ ОСВЕДОМЛЕННОСТИ НА ПРИМЕРЕ МОДИФИЦИРОВАННОЙ ЗАДАЧИ ЭЙНШТЕЙНА

Перечень ключевых слов: ситуационная осведомленность, метрики осведомленности, математическая модель, формальная логика, SMT-solver «Alloy».

Объект исследования – формально-логическая модель модифицированной задачи Эйнштейна с введенными метриками ситуационной осведомленности.

Цель работы – формализация задачи и построение формально-логической модели, позволяющей проверять гипотезы о состоянии модели, а также введение метрик ситуационной осведомленности и формализация задачи ее оптимизации.

Результатом работы является формальная модель вышеописанной задачи, а также программный код, основанный на SMT-solver «Alloy», реализующий эту модель, позволяющий проверять гипотезы о текущем и будущем состоянии модели исходя из возможно неполных данных о ее прошлых состояниях.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ.....	5
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	6
ВВЕДЕНИЕ.....	7
1 Описание решаемой задачи.....	8
1.1 Классическая задача Эйнштейна.....	8
1.2 Модифицированная задача Эйнштейна.....	8
1.3 Описание существующих исследований.....	9
2 Описание формальной модели задачи.....	9
2.1 Обобщенная логическая модель для задач ситуационной осведомленности.....	9
2.2 Модель для модифицированной задачи Эйнштейна.....	10
3 Описание возможных метрик ситуационной осведомленности.....	10
4 Описание программной реализации модели.....	11
4.1 Программный модуль-реализация модели на языке Alloy.....	12
4.1.1 Инициализационные данные.....	12
4.1.2 Служебные сигнатуры.....	12
4.1.2.1 Сигнатура Time.....	13
4.1.2.2 Сигнатура Number.....	13
4.1.2.3 Сигнатура Person.....	13
4.1.2.4 Сигнатура ConstantQualityType.....	13
4.1.2.5 Сигнатура ChangingQualityType.....	13
4.1.2.6 Сигнатура Quality.....	13
4.1.3 Сигнатуры событий.....	14
4.1.3.1 Сигнатура TravellingEvent.....	14
4.1.3.2 Сигнатура MeetingEvent.....	15
4.1.3.3 Сигнатура ExchangeEvent.....	15
4.2 Программный модуль генерации кода на языке Python.....	16
5 Возможное использование программы для модификации агента и ее недостатки.....	16

ЗАКЛЮЧЕНИЕ.....	18
СПИСОК ИСТОЧНИКОВ.....	19
ПРИЛОЖЕНИЯ.....	20

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчете о НИР применяют следующие термины с соответствующими определениями:

SMT-солвер (SMT-solver, SMT-решатель)	Программа, способная проверять совместность набора предикатов, использующих логические связки, кванторы и переменные различных типов.
Агент	Объект, способный взаимодействовать с системой согласно списку установленных правил.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете о НИР применяют следующие сокращения и обозначения:

SA	Ситуационная осведомленность (от англ. Situational Awareness)
DES	Дискретно-событийная симуляция (от англ. Discrete Event Simulation)
API	Интерфейс программного продукта (от англ. Application Programming Interface)

ВВЕДЕНИЕ

Ситуационная осведомлённость – это способность агента (в реальных задачах – человека, компании) воспринимать, анализировать и прогнозировать состояние системы в изменяющихся условиях.

Модель SA, приведенная М. Эндсли в источнике [1] включает три уровня:

- 1) Восприятие (Perception) – способность агента получать и обрабатывать информацию о системе.
- 2) Осмысление (Comprehension) – способность анализировать и оценивать информацию о системы.
- 3) Прогнозирование (Projection) – способность предсказывать будущие состояния системы на основании информации о предыдущих состояниях системы.

Задача SA в таком случае может быть определена как задача оптимизации, с метриками, включающими в себя эти три уровня.

Задача Эйнштейна (также известная как загадка Эйнштейна, головоломка Эйнштейна, загадка о зебре), впервые представленная в источнике [2], представляет собой логическую задачу, в которой необходимо определить принадлежность ряда качеств (например, разнообразных национальностей) ряду объектов (жильцов) на основе набора правдивых утверждений о системе.

Модификация задачи Эйнштейна в данном исследовании заключается в возможности агентов менять свои состояния с течением времени путем обмена качествами.

Результатами работы являются логическая модели острова и ее реализация в виде программы на языке SMT-solver «Alloy», а также оболочка на языке Python, позволяющая автоматически генерировать набор гипотез для проверки на основе логов сторонней программы-симуляции¹ острова.

¹ Источник [10].

1 Описание решаемой задачи

Перед описанием полученной модели опишем решаемую задачу на естественном языке.

1.1 Классическая задача Эйнштейна

В своей первоначальной формулировке задача Эйнштейна представляет собой набор утверждений о некоторой улице из 5 домов, на которой проживает 5 человек различных национальностей, каждый из которых владеет каким-то питомцем, пьет определенный напиток и курит определенную марку сигарет, при этом признаки жильцов уникальны на острове.² Задача решающего – определить взаимное расположение домов, национальность проживающих в этих домах, а также их привычки.

1.2 Модифицированная задача Эйнштейна

В описанную в пункте 1.1 модель вносятся следующие изменения³:

- 1) Состояние системы меняется с течением времени по правилам ниже. Время внутри модели считается дискретным и ограничено некоторым числом. Ходом называется некоторый момент времени.
- 2) Рассматривается не осведомленность стороннего наблюдателя, а осведомленность агентов-жителей острова. Задача агента – иметь максимальную осведомленность об острове.⁴
- 3) Агентам (жителям) разрешено передвигаться между домами, при этом переходы занимают некоторое время. Взаимное расположение домов и расстояния между ними считаются известными изначально для всех жильцов. На нулевом ходу жители находятся в принадлежащих им домах.
- 4) Если два или более человека находятся в одном доме в какой-то момент времени, они считаются встретившимися и узнают всю текущую информацию друг о друге.
- 5) Жители могут обмениваться домами и питомцами во время встречи.
- 6) Жители не могут оставаться в чужих домах дольше одного хода.⁵

² Полная формулировка задачи приведена в источнике [2].

³ Данная формулировка описывает класс задач с различными входными условиями, откуда некоторая расплывчатость формулировок на естественном языке. Эта нечеткость будет формализована в виде набора входных параметров в разделе 4.1.1.

⁴ Варианты формальных метрик описаны в разделе 3.

⁵ Является опциональным правилом и может быть не использовано.

В подобной задаче каждый агент (житель) во время своего хода решает задачу, сходную с классической задачей Эйнштейна, то есть задачей понимания состояния системы на основе частичных данных о ней.

1.3 Описание существующих исследований

Обратим внимание на существующие исследования. Классическая задача Эйнштейна описана и решена в терминах формальной логики (например, в источнике [7]), как и многие модификации этой задачи (источник [6]). Модель для модифицированной задачи Эйнштейна отчасти может опираться на эти модели, однако, вследствие изменчивости состояния системы, модель нуждается в изменении.

Изменения системы могут быть формально заданы, как набор событий, каждое из которых меняет состояние. Такой подход известен, как дискретно-событийная симуляция (DES, discrete event simulation)⁶. Примерами использования DES может быть найден в источнике [8].

2 Описание формальной модели задачи

В этом разделе будет рассматриваться общая модель, основанная на принципах дискретно-событийного моделирования⁷, а также ее уточнение для модифицированной задачи Эйнштейна.

2.1 Обобщенная логическая модель для задач ситуационной осведомленности

Будем рассматривать следующую систему. Модель M представляет собой набор независимых объектов $O = \{O_1, O_2, \dots, O_n\}$. Каждый объект O_i обладает набором состояний $s^i = \{s_1^i, s_2^i, \dots, s_k^i\}$, меняющихся в результате событий. Время в модели будем моделировать набором элементов $T = \{t_1, t_2, \dots, t_n\}$.

События $Q = \{q_1, q_2, \dots, q_m\}$ – это записи об изменениях состояния одного или нескольких объектов. Типы событий будем обозначать, как $E = \{e_1, e_2, \dots, e_t\}$.

Упорядочив события по времени, мы получаем очередь событий, которую можно разбить на уже произошедшие Q_{prev} и будущие Q_{future} .

Также введем функцию текущего состояния объекта:

$$S(O_i, t_j): O \times T \rightarrow S$$

⁶ Описание подхода приведено в источнике [9].

⁷ Приведенная модель является модификацией модели из источника [3].

где $S = \{s^1, s^2, \dots, s^a\}$.

Также вводится множество наблюдателей $W = \{W_1, W_2, \dots, W_n\}$. В соответствие каждому наблюдателю ставится функция, которая возвращает параметры события или объекта.

2.2 Модель для модифицированной задачи Эйнштейна

Опишем модель задачи Эйнштейна в терминах дискретно-событийного моделирования.

Множество O представляет собой множество жителей острова. Состояниями жителей можно считать наборы их признаков (дом, питомец, любимые сигареты и напиток), а также их текущее расположение на острове (нахождение в одном из домов или в пути от одного дома к другому). Время моделируется набором дискретных ходов (шагов).

В системе есть три типа событий: поход в другой дом, обмен качеством с другим жителем, и встреча с другими жителями (несмотря на то, что само состояние объектов не меняется после встречи, меняется осведомленность наблюдателей). Функция текущего состояния будет возвращать множество

$\{\text{дом}_0, \text{питомец}_0, \text{сигареты}_0, \text{напиток}_0, \text{текущее положение}_0\}$.

Множество наблюдателей совпадает с множеством жителей (имеющих ограниченную информацию о состоянии объектов, только о своем состоянии и лично увиденном им во время встреч с другими), объединенного со сторонним наблюдателем (которому известна вся информация об острове).

3 Описание возможных метрик ситуационной осведомленности

В рамках рассмотренной задачи становится возможным вопрос о том, какой стратегии должен придерживаться активный наблюдатель (в нашем случае островитянин), чтобы иметь максимальную осведомленность о ситуации на острове. При этом сама осведомленность может быть оценена по-разному.

Будем рассматривать задачу выбора оптимальной стратегии для двух метрик осведомленности:

- 1) A_1^t – количество известных фактов, отнесенное к общему числу фактов в момент времени t .

- 2) A_2^t – индикаторная функция для утверждения "Мне известно, кому принадлежит зебра в момент времени t ".

Также заметим, что возможны различные способы агрегировать значения метрик осведомленности за все ходы. Будем называть это функцией сбора и обозначать, как $F_x(A_i, n)$, где A_i – выбранная метрика осведомленности, n – общее число ходов.

Рассмотрим несколько возможных функций сбора:

- 1) $F_1(A_i, n) = \sum_{t=1}^n A_i^t$, то есть сумма метрик осведомленности за все ходы
- 2) $F_2(A_i, n) = A_i^n$ – то есть метрика осведомленности на последнем ходу. n
- 3) $F_3(A_i, n) = \sum_{t=1}^n \frac{t}{n} \cdot A_i^t$. Схожа с F_1 , но учитывает новые результаты сильнее, чем старые.

Таким образом, оптимальная стратегия агента A при зафиксированных (F_x, A_i) представляет собой такую стратегию, что при фиксированных действиях других агентов $F_x(A_i, n)$ будет максимальным среди значений для всех возможных стратегий игрока A . Продемонстрируем, что оптимальность стратегии агента зависит от выбранной метрики осведомленности:

Рассмотрим упрощенную модель, в которой существует три островитянина $\{Russian, English, Chinese\}$, изначально живущие в домах $\{Red, Blue, Yellow\}$ и имеющие в питомцах $\{Dog, Cat, Zebra\}$ соответственно. Зафиксируем $n = 2$, расстояния между различными домами = 1, и рассмотрим возможные стратегии агентов для $F_2(M_1, 2)$ и $F_3(M_2, 2)$. Если перебрать возможные действия всех акторов, становится видно, что при метрике $F_2(M_1, 2)$ никакая стратегия поведения не дает преимущества, если считать, что остальные акторы ходят случайно. В то же время для $F_3(M_2, 2)$ стратегии отличаются (так, для *Chinese* оптимальная стратегия на шаге 1 – остаться дома, если считать, что остальные акторы ходят случайно).

Рассмотрение оптимальной стратегии агента выходит за рамки данной статьи, однако продукт проекта может быть использован для ее оптимизации⁸.

4 Описание программной реализации модели

Программная реализация модели состоит из двух частей:

- 1) Программа на языке SMT-солвера Alloy, реализующая собственно модель.

⁸ В разделе 5 рассматривается возможность построения агента, получающего дополнительную информацию за счет анализа ситуации с использованием модели.

- 2) Программа на языке Python, реализующая частичную генерацию кода для модификации модели согласно инициализационным данным.

Реализация доступна по ссылке [Dakirr/Coursework_SA_Alloy: Materials for coursework 2025/2026 on situational awareness, using alloy solver](https://github.com/Dakirr/Coursework_SA_Alloy: Materials for coursework 2025/2026 on situational awareness, using alloy solver).

4.1 Программный модуль-реализация модели на языке Alloy⁹

В этом разделе будет обсуждаться реализация описанной в разделе 2 на языке SMT-солвера «Alloy». Реализация разбита на 4 части для удобства генерации кода программой-оберткой на Python: библиотеку (ядро модели), API, инициализационные данные и набор гипотез для проверки.

Программный код на языке Alloy¹⁰ состоит из двух основных частей: сигнатур (классов объектов, которые могут принимать значения согласно определенным правилам) и фактов (предикатов об объектах сигнатур, которые должны быть тождественно верны). По результатам запуска Alloy будет пытаться построить пример¹¹, в которой выполнены все факты, описанные в тексте программы, и возможны три вывода:

- 1) Пример существует (система совместна),
- 2) Примера не существует (система не совместна),
- 3) Пример не удалось найти за разумное время (система возможно не совместна).

4.1.1 Инициализационные данные

Входными данными модели являются:

- Номер окончательного хода (момента времени) – *final_time* (при *final_time* = 3 в системе будут ходы с номерами {0, 1, 2}).
- Количество жителей острова – *players_num*.
- Списки постоянных и изменяемых признаков – *constant_quality_types* и *changing_quality_types*, соответственно.
- Расстояния между домами на острове – *distances*. По умолчанию между любыми двумя домами расстояние равно 1.

4.1.2 Служебные сигнатуры

⁹ В приложении 1 приведено полное описание задачи из раздела 1.2, формализованное в соответствии с данной моделью.

¹⁰ В этом разделе кратко описываются элементы языка, использованные в конкретной реализации. Более полное описание деталей дизайна языка Alloy можно найти в источнике [4].

¹¹ Под примером здесь понимается экземпляр (instance) модели.

Существует ряд сигнатур, описывающих объекты задачи, но не являющихся событиями. В основном эти объекты описывают состояние системы.

4.1.2.1 Сигнатура Time

Сигнатура Time содержит единственное поле t целочисленного типа и описывает некоторый ход. Существует несколько аксиом, описывающих сигнатуру:

- Для любого t_1 меньшего константы $final_time$ существует единственный экземпляр сигнатуры Time со значением $t = t_1$ (Единственность хода с валидным номером).
- Для любого t_1 большего или равного константе $final_time$ не существует экземпляров сигнатуры Time со значением $t = t_1$ (Несуществование ходов после отметки о финальном ходе).

4.1.2.2 Сигнатура Number

Сигнатура Number является перечислением объектов N_i для всех неотрицательных i от 0 до числа агентов. Является порядковым номером признака или агента и нужна для сокращения числа расчетов, связанных с натуральными числами.

4.1.2.3 Сигнатура Person

Сигнатура Person содержит единственное поле id типа Number, и является обозначением агента в системе. Для сигнатуры Person существует аксиома:

- Для любого N – экземпляра Number существует единственный экземпляр Person с id , равным N (Единственность агента с валидным номером в системе).

4.1.2.4 Сигнатура ConstantQualityType

Сигнатура ConstantQualityType является перечислением возможных неизменяемых признаков агентов. В формулировке задачи из 1.2 таковыми являются, например, любимые напитки и марка сигарет.

4.1.2.5 Сигнатура ChangingQualityType

Сигнатура ChangingQualityType является перечислением возможных изменяемых признаков агентов. В формулировке задачи из 1.2 таковыми являются питомцы и дома агентов.

4.1.2.6 Сигнатура Quality

Сигнатура Quality описывает факт обладания агентом значением признака в данный момент времени (например, в формулировке задачи из 1.2 качеством могло бы выражаться высказывание «Англичанину принадлежит желтый дом на 1 ходу»). Сигнатура содержит поля *person* (сигнатура Person; тот, кому принадлежит признак), *timestamp* (сигнатура Time; о каком ходе данная запись), *type* (сигнатура QualityType¹²; о каком типе признака идет речь), *value* (сигнатура Number; о каком значении признака идет речь). Например, вышеупомянутое высказывание могло бы представляться в виде экземпляра *q* сигнатуры Quality со значениями:

- *person* = Person [*id* = N_1] (при условии, что житель с таким *id* – англичанин)
- *timestamp* = Time [*t* = 1]
- *type* = House (при условии, что House ∈ QualityType)
- *value* = N_1 (при условии, что N_1 – номер желтого дома)

Существует несколько аксиом для данной сигнатуры:

- Каждым значением каждого признака в любой момент времени владеет ровно один агент (существует ровно один объект Quality с [*type*₀, *value*₀, *time*₀]).
- Каждый агент в любой момент времени владеет ровно одним признаком каждого типа (существует ровно один объект Quality с [*type*₀, *person*₀, *time*₀]).
- В начальный момент времени агент с *id* = N_i обладает значениями всех признаков, имеющих номер N_i .
- Неизменяемые качества не меняются.

4.1.3 Сигнатуры событий

События – это сигнатуры, описывающие факт изменения в системе. Выделено три типа событий: путешествия, встречи и обмены.

4.1.3.1 Сигнатура TravellingEvent

Сигнатура TravellingEvent описывает факт путешествия агента из дома в другой. Содержит поля *start* и *arrival* (сигнатура Time; время ухода и прихода, соответственно), *person* (сигнатура Person; идущий агент), *from* и *to* (сигнатура Number, номера домов начала и конца путешествия, соответственно). Для этой сигнатуры существует ряд аксиом:

¹²Является объединением множеств значений ChangingQualityType и ConstantQualityType.

- Ни в какой момент времени агент не может одновременно находиться в двух путешествиях.
- Дома отправления и прибытия не совпадают.
- Время путешествия равно расстоянию между домами или путешествие не завершается ($arrival.t = end_time - 1$).
- Агент находится в доме¹³, из которого уходит.
- Путешествие не может начинаться в нулевой ход.
- Агенты не могут оставаться в не принадлежащим им доме дольше одного хода.

4.1.3.2 Сигнатура MeetingEvent

Сигнатура MeetingEvent описывает факт встречи нескольких агентов в одном доме. Содержит поля *time* (сигнатура Time, время события), *people* (непустое множество сигнатур Person; участники встречи), *house* (сигнатура Number; дом, в котором происходит встреча). Для этой сигнатуры существует ряд аксиом:

- Любой TravellingEvent заканчивается¹⁴ ровно одним MeetingEvent.
- Для любого MeetingEvent существует хотя бы один TravellingEvent, к нему приведший¹⁵.
- Каждый человек может участвовать не более, чем в одном MeetingEvent в ход.
- В каждом доме происходит не более одного MeetingEvent в ход.
- Все люди в доме, в котором происходит MeetingEvent, участвуют в нем.

4.1.3.3 Сигнатура ExchangeEvent

Сигнатура ExchangeEvent описывает событие односторонней передачи качества другому человеку. Множество последовательных взаимных обменов одним и тем же признаком, произошедших за один ход, можно представить, как одновременную передачу всеми участниками признака ровно одному другому агенту.

Доказательство: рассмотрим множество агентов $A_1, A_2, \dots, A_n$, участвующих в обмене, и принадлежащих им признаков $q_1, q_2, \dots, q_n$. Рассмотрим некоторое i . В результате обмена качеством q_i стал обладать некоторый агент A_j . Рассмотрим два случая:

- 1) $i = j$. Тогда агента A_i можно считать не участвовавшим в обмене.

¹³ Описывается предикатом IsInHouse, детали его реализации можно увидеть в программном коде.

¹⁴ Связь TravellingEvent и MeetingEvent определяется как совпадение дома и времени события встречи с временем и домом прибытия TravellingEvent.

¹⁵ Аналогично предыдущему.

2) $i \neq j$. Тогда зафиксируем это как передачу качества от агента A_i к агенту A_j .

Проделав подобное преобразование для всех i , получим, что для всех участвовавших в обмене существует единственная передача от агента другому. Они образуют один или несколько циклов передач.

В рамках реализации любой обмен будем представлять таким образом.

Для сигнатуры существуют следующие аксиомы:

- Человек не может меняться сам с собой.
- Обмен происходит только по итогам MeetingEvent.
- Если есть передача от агента i , то в тот же ход и с тем же типом качества есть передача агенту i .
- Существует не более одной передачи от агента i в ход одного типа качества.

4.2 Программный модуль генерации кода на языке Python

Программный модуль на языке Python необходим для более широких возможностей в настройке модели, например для указания количества агентов.

Представляет из себя генератор кода языка Alloy, который в дальнейшем запускается как часть модели.

Этот модуль также позволяет интегрироваться со сторонней программой-симуляцией острова (источник [10]) путем чтения выходных логов программы, в частности с записями о знаниях конкретного агента, тем самым открывая возможность проверки гипотез «от его лица», что дает возможность более эффективно использовать полученную информацию для выбора дальнейшей стратегии.

5 Возможное использование программы для модификации агента и ее недостатки

Полученный программный модуль можно использовать для проверки гипотез о модели на основе неполных данных. Это позволяет использовать его внутри агента, принимающего решения о дальнейших действиях, дополняя его систему знания (то есть дает ему возможность более полно оценивать и прогнозировать состояние модели. При этом возникает две основные проблемы:

- 1) Производительность. Поскольку сложность вычисления растет экспоненциально параметрам модели¹⁶, модель крайне чувствительна к увеличению параметров (таких, как количество жителей или максимальное время).
- 2) Неполнота. Текущая программа симуляции, являющаяся сторонним ресурсом, не позволяет составлять полную базу знаний агента, поскольку информация о MeetingEvent не содержит информации о текущих качествах участников.

Первую проблему возможно отчасти решить созданием дополнительного модуля «внутренней симуляции» агента, поддерживающей его прошлые знания о модели. В таком случае необходимо поддерживать информацию не обо всей истории изменений, а об изменениях за последние k ходов.

Вторая проблема требует доработки формата логов сторонней модели для включения в них информации о текущем состоянии участников.

При решении двух проблем выше будет возможно пытаться «достраивать» знания агента об острове путем проверки соответствующих гипотез (так, объединяя всю известную информацию об острове с некоторым предикатом P и проверяя полученную модель, вердикт о совместности (несовместности) модели можно использовать, как вердикт об истинности (ложности) предиката. Это увеличивает осведомленность агента, что и является целью оптимизации.

¹⁶ Согласно источнику [5].

ЗАКЛЮЧЕНИЕ

В рамках данной НИР была рассмотрена модификация задачи Эйнштейна, и построена ее формально-логическая модель. Программная реализация модели позволяет проверять гипотезы о текущем и будущем состоянии системы на основе частичных данных о модели, что может быть использовано для моделирования взаимодействия агентов в задачах ситуационной осведомленности, а также для оптимизации стратегий агентов.

Подобные модели могут быть использованы для симуляции реальных событий, и открывают возможности для оптимизации поведения агентов в прикладных задачах. При этом собственно задача ситуационной осведомленности может быть использована для оптимизации методов обмена информацией между агентами.

СПИСОК ИСТОЧНИКОВ

1. Endsley, Mica R. Toward a theory of situation awareness in dynamic systems / Mica R Endsley // Human factors. — 1995. — Vol. 37, no. 1. — Pp. 32–64.
2. Are You a Logical Thinker? Here's a Test // Life International. — 1962. — Vol. 33, no. 13. — P. 92.
3. Хельвас, А.В. & Галицкий, А.С. (2019). Практическое применение языка Alloy для распознавания тактических ситуаций.
4. Jackson D. Logic, language and analysis. The MIT Press, 2006. 369 p
5. Daniel Jackson. 2019. Alloy: a language and tool for exploring software designs. Commun. ACM 62, 9 (September 2019), 66–76. <https://doi.org/10.1145/3338843>
6. Groza, Adrian. Einstein Puzzles / Adrian Groza // Modelling Puzzles in First Order Logic. — Cham: Springer International Publishing, 2021. — Pp. 103–130. https://doi.org/10.1007/978-3-030-62547-4_6.
7. Goliney, Igor. (2015). Solving zebra puzzle with Boolean algebra. 10.13140/RG.2.1.1645.6807.
8. Дискретное моделирование процесса восстановительного ремонта участка дороги / Е.В. Мелешко, Т.С. Афанасенко, Ш.М. Гаджимирзаев et al. // Компьютерные исследования и моделирование. — 2022. — Vol. 14. — P. 1255–1268. <https://doi.org/10.20537/2076-7633-2022-14-6-1255-1268>.
9. Fishman, George S. Discrete-event simulation / George S. Fish man. — Berlin, Heidelberg: Springer-Verlag, 2001
10. cathlx (Catherine). Dynamic Zebra Puzzle Island Simulation: репозиторий GitHub. — URL: https://github.com/cathlx/zebra_puzzle_island (дата обращения: 24.04.2026).

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ 1

Приведем пример значений инициализационных переменных на примере класса задач, описанных в разделе 2:

- $Number = \{N_0, N_1, N_2, N_3, N_4, N_5\}$
- $ConstantQualityType = \{Sigarettes, Drink, Nationality\}$
- $ChangingQualityType = \{House, Pet\}$

Приведем соотнесение значений Number значениям на естественном языке:

Таблица 1. Соотнесение значений признаков на естественном языке с их нумерацией.

<i>value</i>	<i>Sigarettes</i>	<i>Drink</i>	<i>Nationality</i>	<i>House</i>	<i>Pet</i>
N_0	<i>Pall Mall</i>	<i>Whiskey</i>	<i>German</i>	<i>Green</i>	<i>Cat</i>
N_1	<i>Dunhill</i>	<i>Beer</i>	<i>Russian</i>	<i>Blue</i>	<i>Fish</i>
N_2	<i>Kent</i>	<i>Juice</i>	<i>Chinese</i>	<i>Yellow</i>	<i>Zebra</i>
N_3	<i>Camel</i>	<i>Water</i>	<i>French</i>	<i>White</i>	<i>Dog</i>
N_4	<i>Parlament</i>	<i>Vodka</i>	<i>American</i>	<i>Red</i>	<i>Bear</i>
N_5	<i>Marlboro</i>	<i>Wine</i>	<i>English</i>	<i>Black</i>	<i>Hamster</i>